香港科技大学(广州)
THE HONG KONG UNIVERSITY OF SCIENCE AND TECHNOLOGY (GUANGZHOU)

Improving Text2SQL through Query Planning and Semantic Back-Verification

DSAA6000R (L01) - Data Intelligence

Group 8: Talk2DB

Enhui Yang
Yingjin Jiang
Xiaoyan Su
Zhengxi Shao
Zhengquan Li

Contents

1 Introduciton **1**

2 Problem Description **1**

3 Methodology **2**
3.1 Framework Overview 2
3.2 SQL Generation 2
3.3 SQL2TextValidator 2
3.4 SQL Checker 3
3.5 Experiment Environment 3

4 Measurement **3**
4.1 Evaluation Setup 3
4.2 Evaluation Metrics 3
4.3 Main Results 4

5 Analysis and Discussion **4**
5.1 Effectiveness on Complex Queries 4
5.2 Limitations and Failure Cases 5

6 Conclusions **5**

1 Introduciton

This report aims to improve the accuracy and reliability of current Text-to-SQL (Text2SQL) systems. Given a natural language query, the goal of a Text2SQL model is to generate a concise and semantically correct Structured Query Language (SQL) query that retrieves the desired result from the target database. In this work, we first constructed a basic ReAct-based agent following previous studies [4, 5, 11] and evaluated it on standard Text2SQL benchmark [12]. Through analysis of the failure cases, we identified two major weaknesses that have not been adequately addressed in prior research. To tackle these issues, we propose Talk2DB, a framework designed to provide effective solutions to the identified limitations. This framework incorporates a schema linker, a content retriever, and a semantic validator to improve SQL generation accuracy through both a preprocessing stage and a post-generation validation step.

In Section 2, we describe the current limitations and discuss how they lead to failures in the test cases. Building on this analysis, we outline several potential approaches to address these issues. Section 3 presents our proposed methods in detail. In Section 4, we evaluate our framework using mainstream benchmark, and the result demonstrate that it achieves improved performance compared with baseline models by 16.13% and 21.70%.

2 Problem Description

Text-to-SQL has become an active research topic in recent years. This task takes a natural language (NL) question as input and produces an SQL query capable of retrieving the correct subset of data. With the rapid development of large language models (LLMs), great progress has been made in improving SQL generation accuracy. However, through our examination of existing research [9, 3, 5] and the application of our benchmarked ReAct-based model to two widely used benchmarks, we identified two notable limitations that remain unaddressed.

- **The lack of query planning.** Previous research utilize LLMs to direct generate SQL queries from NL question, leading to result that these SQL queries are syntactically valid but do not match the actual semantic meanings of the quesitons. The accuracy of the generation remains unsatisfied as complex questions requires more measures, including schema linking, subproblem decomposition, and so on.

- **Limited Techniques for Validating SQL Semantic Correctness.** Due to the inherent ambiguity of natural language, LLMs may misinterpret user intent during the translation of natural-language questions into SQL queries, resulting in semantic misalignment between the NL question and the generated SQL. A query may be syntactically valid and executable, yet still fail to capture the intended semantics of the original question. In such cases, simplistic validation method, such as syntactic checks or execution-based correctness are insufficient.

To address the above limitations, we propose Talk2DB, a novel framework that integrates a schema linker, a content retriever, and a semantic validator to enhance the SQL generation process. The schema linker and content retriever provide fine-grained contextual information, enabling more accurate query planning and interpretation. The validator further ensures correctness by translating the generated SQL back into natural language and comparing it with the original user query to detect semantic inconsistencies. Experiments on the Spider benchmark [12] demonstrate that our approach achieves strong performance, validating the effectiveness of the proposed framework.

3 Methodology

3.1 Framework Overview

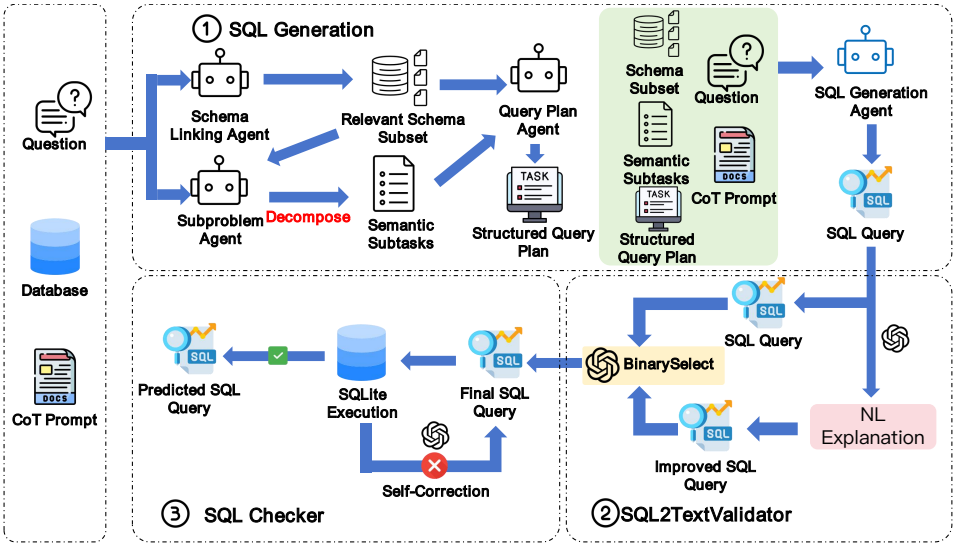

Figure 1: The framework of Talk2DB.

To enhance the stability and reliability of SQL query generation, we propose a robust three-stage iterative framework. Our framework is illustrated in Figure. 1. In the first stage, we use multiple agents to generate related context for the final query generation. This helps the LLMs gather the relevant information and minimizes the range of related tables and columns. Then, in the next stage, the generated queries will be validated through a semantic check process. The LLM translates the SQL queries into NL questions and checks the semantic meanings between the original and the translated questions, further ensuring their consistency. Finally, the SQL queries will be executed on the real database to check the syntactic correctness. If the queries pass the check, they will be output as the final answer. Otherwise, the error messages from the system will be used as feedback to help the LLM refine the queries.

3.2 SQL Generation

This stage aims to output reliable SQL queries by decomposing complex questions into subproblems and selecting related tables and columns before generation. The schema linking agent employs a CoT prompt to select a subset of relevant schema from the original database. Then the selected schema serves as context for both the query plan agent and the subproblem agent. The subproblem agent decomposes the natural language questions into multiple subproblems. The subproblems, relevant schema, and the natural language questions are utilized by the query plan agent to generate a structured query plan for guiding the generation, as shown in Figure. 1. Then the SQL generation agent produces the SQL queries based on the plan and the related context.

3.3 SQL2TextValidator

As the LLMs may misunderstand the actual meaning of the natural language questions, we employ an extra step to translate the SQL queries back into natural language questions. This method has been proven to be effective in previous research [2] since LLMs are trained on a large scale of natural language

corpus. The LLM compares the semantic meanings of the original natural language questions and the translated questions from the SQL queries. If the translated questions are not consistent with the original ones, another generation round will be launched to try to produce a different SQL query. This iterative process will stop when the translated questions are considered semantically the same as the original natural language questions.

3.4 SQL Checker

While the previous stage ensures semantic correctness, the syntax of the generated SQL queries remains to be validated. In this phase, we verify syntactic validity by executing the queries against the target database. Queries that execute successfully and yield valid results are designated as the final output. Conversely, if execution triggers an error or returns an empty result, this feedback is incorporated into the prompt to guide the next generation iteration.

3.5 Experiment Environment

We set up our experiment on a MacBook Pro M4 running macOS 15.6. The study employs Qwen2.5-7B [10, 7] as the base LLM. We use LangGraph [8] to construct our framework, enabling the LLM to follow a ReAct [11] pattern to refine the original SQL queries. To balance response diversity and precision, the sampling temperature is fixed at 0.6. The maximum number of revision rounds in the third stage is limited to three, consistent with the baseline configuration.

4 Measurement

4.1 Evaluation Setup

We evaluate our framework using the widely adopted Spider benchmark [12], a large scale Text to SQL dataset consisting of natural language questions paired with SQL queries across diverse domains. Following prior work [6, 1, 13], all experiments are conducted in a zero-shot setting without any task-specific fine-tuning. To ensure comparability across systems, we adopt consistent execution limits, timeout thresholds, and revision budgets. Each framework is allowed up to three iterative refinement rounds, and all evaluations are performed using identical sampling and decoding parameters as described in Section 3. To evaluate the specific contribution of our full pipeline, we implemented a ReAct-based baseline that exclusively incorporates the third stage for SQL query refinement. This baseline serves as a comparative reference to validate the effectiveness of our proposed multi-stage approach.

4.2 Evaluation Metrics

To comprehensively assess both the executability and efficiency of our framework, we employed two evaluation metrics from the spider dataset:

- **Exact Set Match Accuracy (EM).** This metric measures whether the generated SQL queries structurally match the ground-truth queries. To account for ordering differences, each clause is treated as a set. A generated query is considered correct only when all its components exactly match those of the ground truth.
- **Execution Accuracy (EX).** This metric compares the execution results fetched from the generated queries against those of the ground-truth queries. It treats the generated queries as correct when the returned results are identical.

4.3 Main Results

Table 1 summarizes the evaluation results of our experiments in terms of EM and EX. Our complete framework outperforms the baseline by 21.42% in EX, with the first stage (Query Plan) contributing more significantly than the backtranslation stage. While each component has a positive impact on the overall EX performance, we observe a slight decrease in EM when the Backtranslation component is absent (i.e., the variant utilizing only Query Plan). We attribute this to the fact that the Query Plan tends to generate semantically correct but structurally complex queries that deviate from the ground truth, leading to non-standard clauses. Conversely, the Query Plan and Subproblem Decomposition make a substantial contribution to the framework, yielding an increase of 18.04% compared to the baseline. This demonstrates the effectiveness of our strategy, particularly in handling complex questions.

Table 1: Ablation study of different components in the Talk2DB framework on the Spider development set. We report the **Execution Accuracy (EX)** across all difficulty levels and the **Exact Match (EM)** accuracy on the overall set. The "Improvement" denotes the relative gain in Execution Accuracy (All) compared to the baseline.

Framework Variants	**Execution Accuracy (EX)**					**Exact Match (All)**	**Improvement (EX)**
	Easy	**Medium**	**Hard**	**Extra**	**All**		
Baseline (ReAct only)	0.885	0.541	0.506	0.294	0.593	0.580	-
Talk2DB w/o BackTranslate	0.833	0.550	0.571	0.471	0.620	0.553	+4.55%
Talk2DB w/o Query Plan	0.846	0.658	0.610	0.706	0.700	0.640	+18.04%
Talk2DB (Full)	0.846	0.658	0.649	0.794	**0.720**	**0.673**	**+21.42%**

5 Analysis and Discussion

This section analyzes the experimental results and provides further discussion to elucidate the mechanisms underlying Talk2DB.

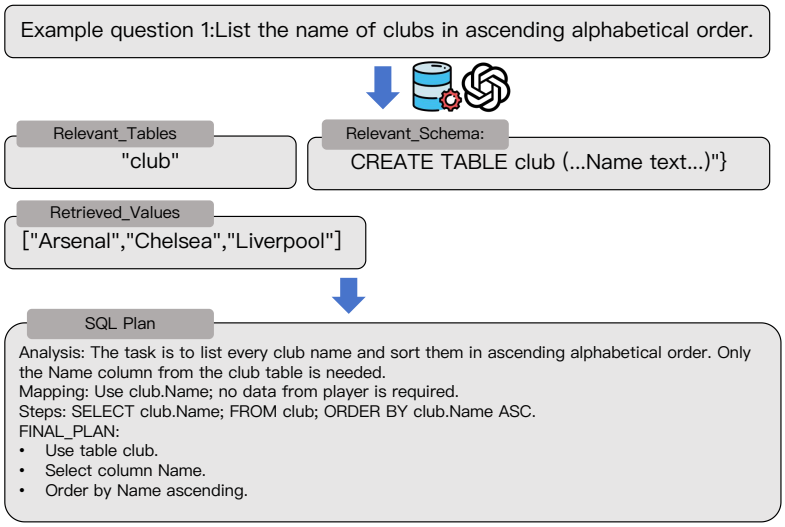

Figure 2: An example to illustrate how the Query Plan provide effective context for SQL generation.

5.1 Effectiveness on Complex Queries

As illustrated in Figure 2, Talk2DB initiates the process by extracting relevant schema and information from the original database to serve as context. Subsequently, a Query Plan Agent uses this context to

formulate a structured plan, generating each component of the query before composing them into the final answer. Through subproblem decomposition and query planning, complex NL inquiries are transformed into a set of simpler subproblems. This strategy effectively mitigates the inference limitations often observed in smaller-scale LLMs. Following generation, the validator translates the SQL query " SELECT Name FROM club ORDER BY Name ASC" back into a natural language question (e.g., "What are all the club names, listed in ascending alphabetical order?") and checks for semantic consistency with the original user query. This step further validates the correctness of the generated SQL.

5.2 Limitations and Failure Cases

Although Talk2DB achieved significant improvements in both execution accuracy and EM rate compared to the ReAct baseline model, the failure cases on the Spider development set still expose two limitations that need to be urgently addressed.

First, many hard and extra-level failure cases can be attributed to incomplete reasoning over multi-hop join paths in schemas that contain several bridge tables. For example, in e-commerce databases, the model sometimes joins Products directly with Shipments while omitting the necessary Shipment_Items relation, or applies filters defined at the order level to shipment-level attributes. Similar patterns appear in the transportation and education domains, where the generated queries select an underspecified join chain or attaches constraints to an incorrect relation. These issues suggest that although schema linking reduces the search space, the current query planner still struggles to model multi-relational dependencies with sufficient fidelity.

Second, the SQL2TextValidator is not always sensitive to fine-grained logical differences. Several failure cases differ from the gold query only in inequality directions (`>` vs. `>=`), numeric thresholds (e.g., `> 2` vs. `>= 3`), or the choice between `IN` and `NOT IN`. Such discrepancies can still yield plausible natural-language paraphrases, which are judged semantically consistent by the validator and therefore allowed to pass. Moreover, for a few highly compositional questions, the iterative refinement process hits the pre-defined timeout or revision budget and degenerates into trivial outputs (e.g., `SELECT 1;`), exposing a trade-off between robustness and computational efficiency. These limitations suggest that future work should incorporate more fine-grained logical checking and stronger guidance for join reasoning and set operations.

These failure cases highlight that future work should prioritize improving aggregation and set-operator fidelity, while developing more reliable mechanisms for modeling complex schema topologies and multi-hop relational reasoning.

6 Conclusions

We proposed Talk2DB, which integrates Subproblem Decomposition and Query Planning to tackle the challenges of complex query generation, followed by a backtranslation stage for semantic validation. The empirical results indicate that our approach significantly surpasses the baseline, yielding a 21.42% gain in EX. Moving forward, we aim to explore more robust optimization strategies and leverage next-generation LLMs to continue refining the framework.

References

[1] Zhenbiao Cao, Yuanlei Zheng, Zhihao Fan, Xiaojin Zhang, Wei Chen, and Xiang Bai. Rsl-sql: Robust schema linking in text-to-sql generation. *arXiv preprint arXiv:2411.00073*, 2024.

[2] Daojun Chen, Xi Wang, Shenyuan Ren, Qingzhi Ma, Pengpeng Zhao, and An Liu. Gbv-sql: Guided generation and sql2text back-translation validation for multi-agent text2sql. *arXiv preprint arXiv:2509.12612*, 2025.

[3] Dawei Gao, Haibin Wang, Yaliang Li, Xiuyu Sun, Yichen Qian, Bolin Ding, and Jingren Zhou. Text-to-sql empowered by large language models: A benchmark evaluation. *arXiv preprint arXiv:2308.15363*, 2023.

[4] Mohammadreza Pourreza, Hailong Li, Ruoxi Sun, Yeounoh Chung, Shayan Talaei, Gaurav Tarlok Kakkar, Yu Gan, Amin Saberi, Fatma Ozcan, and Sercan O Arik. Chase-sql: Multi-path reasoning and preference optimized candidate selection in text-to-sql. *arXiv preprint arXiv:2410.01943*, 2024.

[5] Mohammadreza Pourreza and Davood Rafiei. Din-sql: Decomposed in-context learning of text-to-sql with self-correction. *Advances in Neural Information Processing Systems*, 36:36339–36348, 2023.

[6] Shayan Talaei, Mohammadreza Pourreza, Yu-Chen Chang, Azalia Mirhoseini, and Amin Saberi. Chess: Contextual harnessing for efficient sql synthesis. *arXiv preprint arXiv:2405.16755*, 2024.

[7] Qwen Team. Qwen2.5: A party of foundation models, September 2024.

[8] Jialin Wang and Zhihua Duan. Empirical research on utilizing llm-based agents for automated bug fixing via langgraph. *arXiv preprint arXiv:2502.18465*, 2025.

[9] Zhongyuan Wang, Richong Zhang, Zhijie Nie, and Jaein Kim. Tool-assisted agent on sql inspection and refinement in real-world scenarios. *arXiv preprint arXiv:2408.16991*, 2024.

[10] An Yang, Baosong Yang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Zhou, Chengpeng Li, Chengyuan Li, Dayiheng Liu, Fei Huang, Guanting Dong, Haoran Wei, Huan Lin, Jialong Tang, Jialin Wang, Jian Yang, Jianhong Tu, Jianwei Zhang, Jianxin Ma, Jin Xu, Jingren Zhou, Jinze Bai, Jinzheng He, Junyang Lin, Kai Dang, Keming Lu, Keqin Chen, Kexin Yang, Mei Li, Mingfeng Xue, Na Ni, Pei Zhang, Peng Wang, Ru Peng, Rui Men, Ruize Gao, Runji Lin, Shijie Wang, Shuai Bai, Sinan Tan, Tianhang Zhu, Tianhao Li, Tianyu Liu, Wenbin Ge, Xiaodong Deng, Xiaohuan Zhou, Xingzhang Ren, Xinyu Zhang, Xipin Wei, Xuancheng Ren, Yang Fan, Yang Yao, Yichang Zhang, Yu Wan, Yunfei Chu, Yuqiong Liu, Zeyu Cui, Zhenru Zhang, and Zhihao Fan. Qwen2 technical report. *arXiv preprint arXiv:2407.10671*, 2024.

[11] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. React: Synergizing reasoning and acting in language models. In *ICLR 2023*, 2023.

[12] Tao Yu, Rui Zhang, Kai Yang, Michihiro Yasunaga, Dongxu Wang, Zifan Li, James Ma, Irene Li, Qingning Yao, Shanelle Roman, et al. Spider: A large-scale human-labeled dataset for complex and cross-domain semantic parsing and text-to-sql task. *arXiv preprint arXiv:1809.08887*, 2018.

[13] Yuxin Zhang, Meihao Fan, Ju Fan, Mingyang Yi, Yuyu Luo, Jian Tan, and Guoliang Li. Reward-sql: Boosting text-to-sql via stepwise reasoning and process-supervised rewards. *arXiv preprint arXiv:2505.04671*, 2025.